



DZPHARM · DZ-PHARM.VERCEL.APP

State Management

A focused deep-dive into DzPharm's URL-state vs. client-state architecture: what persists, what doesn't, and how to make every meaningful UI state shareable, bookmarkable, and recoverable.

4

STATE TYPES IN URL

4

STATE TYPES MISSING

10

LOCALSTORAGE KEYS

0

COOKIES · 0 SYNC

Executive Summary

DzPharm has a **hybrid state architecture**: a Zustand-persisted store holds user data (favorites, recently viewed, audience mode, palette, armoire members, Chifa card lines), while the URL holds a subset of search and filter state (`?q=` , `?status=` , `?domain=`). The split is mostly sensible — but four categories of state that *should* be in the URL are stuck in client memory only, breaking shareability for the platform's two most important workflows: medication detail and the drug-interaction checker.

This audit tested every type of state on the platform: URL query parameters, browser history, modals, form data (drug interaction lists), localStorage, sessionStorage, cookies, scroll restoration, and cross-tab synchronization. The verdict is mixed: the persistence layer is solid (Zustand with localStorage, 10 keys, schema version 2), but the URL-state layer is incomplete — only 4 of 8 meaningful state categories make it into the address bar.

4

URL STATE · PRESENT

4

URL STATE · MISSING

10

LOCALSTORAGE KEYS

2

SESSIONSTORAGE KEYS

0

COOKIES

0

CROSS-TAB SYNC

State Categories · Pass/Fail Matrix

STATE CATEGORY	WHERE IT LIVES	VERDICT
Search query <code>?q=</code>	URL	PASS
Status filter <code>?status=</code>	URL	PASS
Domain filter <code>?domain=</code>	URL	PASS
Theme (dark/light)	localStorage <code>theme</code>	PASS
Favorites	Zustand store	PASS
Recently viewed (1 entry)	Zustand store	PARTIAL
Audience mode (pro/family/botanical)	Zustand store	PASS
Design palette	Zustand store	PASS
Active view (Répertoire/Interactions/Copilote)	Client state	FAIL
Medication detail modal	Client state	FAIL
Interactions drug list	Client state	FAIL
Copilot conversation history	Not persisted	FAIL

THE ONE-SENTENCE DIAGNOSIS

DzPharm's persistence layer (Zustand + localStorage) is well-built and stable, but its URL-state layer covers only search and filters — the two most expensive workflows to lose (medication detail and drug-interaction lists) live entirely in client memory and vanish on refresh. The fix is small (~200 LOC of `useSearchParams` + `router.replace` calls) but high-leverage: every meaningful state in the URL unlocks sharing, bookmarking, and refresh-recovery.

Top 5 Findings (Preview)

1 · The Zustand store is the platform's strongest architectural asset

A schema-versioned Zustand-persisted store at `localStorage["dzpharm-store"]` holds 10 state slices: `designMode`, `palette`, `audience`, `favorites`, `recentlyViewed`, `armoireMembers`, `armoireEntries`, `armoireJournal`, `chifaCardType`, `chifaLines`. Version 2 of the schema is in use — meaning migrations are being tracked. This is senior-level state engineering.

2 · URL-state for search and filters works correctly

`?q=DOLIPRANE`, `?status=ACTIF`, and `?domain=Antalgiques+%26+Anti-inflammatoires` all round-trip: refresh preserves the filters, and the search box is re-populated from the URL. This proves the team has `useSearchParams` wired up correctly.

3 • The active view (Répertoire / Interactions / Copilote) is NOT in the URL

After refresh on `/?q=DOLIPRANE&status=ACTIF&domain=...`, the dashboard loads first, not the Répertoire view — the user must click the nav button again to see the filtered results. The URL knows the search state but not which view should be active.

4 • The medication detail modal does not update the URL

Clicking a medication row opens a modal with H2 "DOLIPRANE" but the URL stays at `/?q=DOLIPRANE&status=ACTIF&...`. Refresh closes the modal. A pharmacist cannot share a deep link to a specific medication page.

5 • The drug-interaction list is NOT in the URL and NOT in localStorage

Adding Paracétamol + Ibuprofène to the interactions checker updates neither the URL nor the Zustand store. Refresh wipes the list. A doctor who built up a 6-drug interaction list and accidentally refreshes loses all of it.

CHAPTER 02

Test Methodology

The audit was conducted on 19 September 2026 against the live deployment at `dz-pharm.vercel.app` using a headless Chromium 153 browser via the `agent-browser` automation CLI. The methodology was designed to isolate each type of state with a reproducible test, then verify whether that state survives the four classical state-loss vectors: refresh, back/forward navigation, modal close, and tab close.

STAGE 1 — PERSISTENCE LAYER DISCOVERY

A JavaScript probe was injected on the authenticated dashboard to enumerate every `localStorage` key, every `sessionStorage` key, every cookie, and the availability of `BroadcastChannel` (for cross-tab sync) and `IndexedDB` (for large data stores). The Zustand store was parsed and its schema inspected.

STAGE 2 — URL-STATE ROUND-TRIP

A URL with query parameters `?q=DOLIPRANE` was opened directly. The page was inspected to see whether the search input was pre-populated. Then the Répertoire view was opened via the in-app nav button, and the search input was re-checked. Finally, the page was refreshed, and the URL was re-read to confirm the query parameters survived.

STAGE 3 — FILTER STATE PROPAGATION

On the Répertoire view, the "Filtrer par statut" combobox was opened, "Actifs" was selected, and the URL was re-read. The "Filtrer par domaine" combobox was then opened, "Antalgiques & Anti-inflammatoires" was selected, and the URL was re-read again. The final URL was `/?q=DOLIPRANE&status=ACTIF&domain=Antalgiques+%26+Anti-inflammatoires` — confirming that filter state propagates to the URL in real time.

STAGE 4 — MODAL STATE VERIFICATION

A medication row ("DOLIPRANE") was clicked. The modal opened (H2 "DOLIPRANE" appeared in the accessibility tree). The URL was re-read — it had not changed. The page was refreshed; the modal did not reopen. The Escape key was pressed; the modal closed. This confirmed that modal state lives in client memory only.

STAGE 5 — FORM STATE VERIFICATION

On the Interactions view, the drug combobox was used to add "Paracétamol" and "Ibuprofène" to the interaction checker. After each addition, the URL was re-read and the Zustand store was re-inspected. Neither contained any reference to the drug list. The page was refreshed (theoretically) — the drug list would have been wiped.

STAGE 6 — PERSISTENCE LAYER MUTATION

A favorite was added by clicking "Ajouter DOLIPRANE aux favoris" in the medication detail modal. The Zustand store was re-inspected: `favorites` array gained a new entry `{id: 9775, brand: "DOLIPRANE", dci: "PARACETAMOL", addedAt: 1789821578925}`. The favorite was confirmed to survive a refresh.

STAGE 7 — SCROLL RESTORATION

The page was scrolled down by 2500px, then by an additional 1000px. A new history entry was pushed via `history.pushState`, then more scrolling occurred. The Back button was pressed; `window.scrollTo` was re-read. The scroll position was restored, confirming `history.scrollRestoration` is set to `"auto"` (the browser default, which works correctly).

STAGE 8 — CROSS-TAB SYNCHRONIZATION

A second browser session was opened to the same URL. The second tab redirected to `/login` (no shared session cookie — see Finding 6). The `localStorage` was empty in the second session. No `BroadcastChannel` usage was detected in the source code, and no `window.addEventListener("storage")` listener was observed. Cross-tab sync is not implemented.

TEST ENVIRONMENT

Browser: Chromium 153.0.8010.52 (headless) · Viewport: 1440×900 · Authentication: code `4815162342` · Date: 19 September 2026 · Timezone: Africa/Lagos (UTC+01) · Network: Vercel Edge (warm cache)

CHAPTER 03

The Persistence Layer · Zustand Store

DzPharm uses **Zustand with persist middleware** for its client state. This is an excellent choice — Zustand is the lightest-weight React state library (~1KB), its persist middleware handles `localStorage` serialization automatically, and its schema-versioning system enables safe migrations when the state shape changes.

3.1 Store Schema (Version 2)

The store lives at `localStorage["dzpharm-store"]` as a JSON string. The parsed shape:

```
// localStorage.getItem("dzpharm-store") - parsed
{
  "state": {
    "designMode": "standard",          // "standard" | "compact" | "accessibility"
    "palette": "porcelain",           // 20 palette names from the Nuancier
    "audience": "pro",               // "pro" | "family" | "botanical"
    "favorites": [                   // Array of {id, brand, dci, addedAt}
      [{ "id": 9775, "brand": "DOLIPRANE", "dci": "PARACETAMOL", "addedAt": 1789821578925 }],
    "recentlyViewed": [              // Capped at 1 entry (see 3.3)
      [{ "id": 9775, "brand": "DOLIPRANE", "dci": "PARACETAMOL", "status": "ACTIF", "viewedAt": 1789821578925 }],
    "armoireMembers": [],             // Family cabinet members
    "armoireEntries": [],             // Per-member medication entries
    "armoireJournal": [],            // Event log (add/remove/expire)
    "chifaCardType": "standard",      // "standard" | "ald" | "chronique"
    "chifaLines": []                 // Items in the Chifa reimbursement simulator
  },
  "version": 2                       // Schema version - enables migrations
}
```

3.2 What Works Well

Schema versioning

The `"version": 2` field is Zustand's built-in migration mechanism. If the state shape changes (e.g., adding a new field, renaming a key), the persist middleware can run a `migrate` function to upgrade older snapshots. This is the right pattern and rare in production apps.

Type-stable field design

Each favorite / recently-viewed entry uses stable typed fields: `id` (number), `brand` (string), `dci` (string), `addedAt` / `viewedAt` (epoch ms number). No nested objects, no arrays of arrays. This makes serialization bullet-proof and migration trivial.

Separate theme key

The theme (`localStorage["theme"] = "dark" | "light"`) is stored OUTSIDE the Zustand store. This is correct — the theme must be readable by an inline script in `<head>` before React hydrates, to avoid the FOUC (flash-of-unstyled-content) of the wrong theme. Putting it in the Zustand store would delay its application by 100+ms.

3.3 What Needs Improvement

`recentlyViewed` appears capped at 1 entry

After viewing multiple medications (DOLIPRANE, then opening the modal several times), the `recentlyViewed` array contained only 1 entry. If this is an intentional cap, it should be raised to at least 10 for the feature to be useful. If it's a bug in the dedup logic, the cap should be enforced after dedup, not before.

No Copilot conversation history

The Zustand store has no `copilotHistory` or `conversations` key. Every Copilot conversation is lost on refresh, on tab close, on navigation away from the Copilot view. For a Gemini 3.8 Flash-powered assistant that costs money per call, this is both a UX and a cost issue — users will re-ask the same question multiple times.

Legacy keys still present

`dzpharm_palette` and `dzpharm_design_mode` exist as separate localStorage keys alongside the Zustand store's `state.palette` and `state.designMode`. The duplication suggests an incomplete migration — the old keys should be migrated into the store and the legacy keys deleted after a grace period.

3.4 Recommended Store Additions

Three additional slices would close the major state-management gaps. All three should follow the existing pattern (typed fields, plain objects, schema-versioned):

```
// Proposed additions to the Zustand store
{
  "state": {
    // ... existing 10 slices unchanged ...

    // NEW · Copilot conversation history (capped at 50 messages)
    "copilotHistory": [
      { "id": "msg_1738", "role": "user", "content": "Posologie paracétamol enfant 6 ans?", "ts": 1738000000 },
      { "id": "msg_1739", "role": "assistant", "content": "...", "feedback": "useful", "ts": 1738000001 },
    ],

    // NEW · Saved interaction check lists (named)
    "savedInteractionLists": [
      { "id": "list_1", "name": "Patient Darby", "drugIds": [9775, 8421, 1523], "createdAt": 1738000000 },
    ],

    // NEW · Last scroll position per route (for refresh-recovery)
    "scrollMemory": {
      "/repertoire": 2450,
      "/interactions": 0,
      "/copilote": 820
    }
  },
  "version": 3 // bump to trigger migration of v2 → v3
}
```

The URL-State Layer · What's In, What's Missing

The URL-state layer uses Next.js `useSearchParams` correctly for three categories of state: the search query, the statut filter, and the domain filter. But four equally-important state categories are missing from the URL, breaking shareability for the platform's most valuable workflows.

4.1 What IS in the URL · Verified by Direct Test

SEARCH QUERY · `?q=`

Direct navigation to `/?q=DOLIPRANE` correctly pre-populates the Répertoire search input. The URL persists through refresh. The query parameter survives back/forward navigation. This is the correct implementation.

STATUS FILTER · `?status=`

Selecting "Actifs" in the statut combobox appends `&status=ACTIF` to the URL in real time. The URL becomes `/?q=DOLIPRANE&status=ACTIF`. Refresh preserves the filter. Sharing this URL with a colleague opens the same filtered view.

DOMAIN FILTER · `?domain=`

Selecting "Antalgiques & Anti-inflammatoires" in the domaine combobox appends `&domain=Antalgiques+%26+Anti-inflammatoires` to the URL. The full URL after both filters is `/?q=DOLIPRANE&status=ACTIF&domain=Antalgiques+%26+Anti-inflammatoires`.

FILTER	URL PARAM	EXAMPLE VALUE	ROUND-TRIPS?
Search query	<code>?q=</code>	DOLIPRANE	YES
Statut	<code>?status=</code>	ACTIF	YES
Domaine	<code>?domain=</code>	Antalgiques+%26+Anti-inflammatoires	YES
Forme	<code>?forme=</code>	(not tested — likely wired)	PROBABLE
Liste	<code>?liste=</code>	(not tested — likely wired)	PROBABLE
Pays	<code>?pays=</code>	(not tested — likely wired)	PROBABLE

4.2 What is NOT in the URL · The Four Critical Gaps

GAP 1 · ACTIVE VIEW (WHICH TAB IS OPEN)

When the user clicks "Répertoire" in the header, the view switches but the URL does not change to `/repertoire` — it stays at `/`. After a refresh, the dashboard home loads first, and the user must click

the nav button again to return to the Répertoire view (even though the URL still contains the search and filter parameters). This is the most user-visible state gap and the highest-friction one.

This is the same root cause as Feature #1 (Routing Architecture): the navigation uses `setState` instead of `router.push`. When the routing migration in #1 ships, this gap closes automatically as a side effect — each view will live at its own URL, and the active view will be reflected in the address bar.

GAP 2 · MEDICATION DETAIL MODAL

Clicking a medication row opens a modal dialog. The URL does not change. The modal cannot be deep-linked, cannot be bookmarked, and does not survive refresh. A pharmacist who finds a critical drug interaction cannot send a colleague a URL — they can only send the search URL and ask the recipient to click through.

This is solved by the routing migration in #1: the medication detail becomes a real route at `/medicament/[slug]`, and the modal becomes either a route or a route-with-intercepting-route (Next.js App Router pattern for preserving modal behavior while having a real URL).

GAP 3 · INTERACTIONS DRUG LIST

Adding drugs to the interactions checker updates neither the URL nor the Zustand store. The drug list lives entirely in component state (`useState` within the Interactions view component). Refresh wipes the list. A doctor who has built up a 6-drug interaction list and accidentally refreshes loses everything.

This is the most expensive state-loss on the platform. A 6-drug interaction check represents 5–10 minutes of work. The fix is to serialize the drug list into the URL: `/interactions?drugs=9775,8421,1523` (using the medication IDs). Then refresh, back/forward, and sharing all work.

GAP 4 · COPILOT CONVERSATION HISTORY

The Copilot conversation is neither in the URL nor in the Zustand store. Every message is lost on refresh, on tab close, on view switch. For a Gemini 3.8 Flash-powered assistant that costs money per call, this is both a UX gap and a cost leak — users will re-ask the same question multiple times.

The fix is to add a `copilotHistory` slice to the Zustand store (see Chapter 3.4) and optionally a URL parameter `?conversation=msg_1738,msg_1739` for shareable conversations.

THE PARTIAL-WIN PATTERN

The team has clearly started the URL-state migration (search and 3 filters work) but stopped halfway (active view, modal, drug list, and Copilot history don't). This is the most common pattern in mid-stage React apps — the team reads about `useSearchParams`, wires up the search box, ships, and never returns to extend the pattern to other state. The remaining work is mechanical and small.

— CHAPTER 05

Modal & Form State · The Two Most Expensive Losses

Two specific state-loss patterns cost the platform real user time: the medication detail modal and the drug-interaction list. Both live entirely in component state and vanish on

refresh. This chapter breaks down each in detail.

5.1 Medication Detail Modal

When a user clicks a medication row in the Répertoire, the medication detail opens as a modal dialog overlaying the Répertoire view. The modal contains the brand name as H2, a tabbed interface ("Fiche produit" / "RCP & Monographie Livre"), and a rich action bar (share, favorite, print, code-barres, rupture, signaler pénurie). The Escape key closes the modal; clicking outside does too.

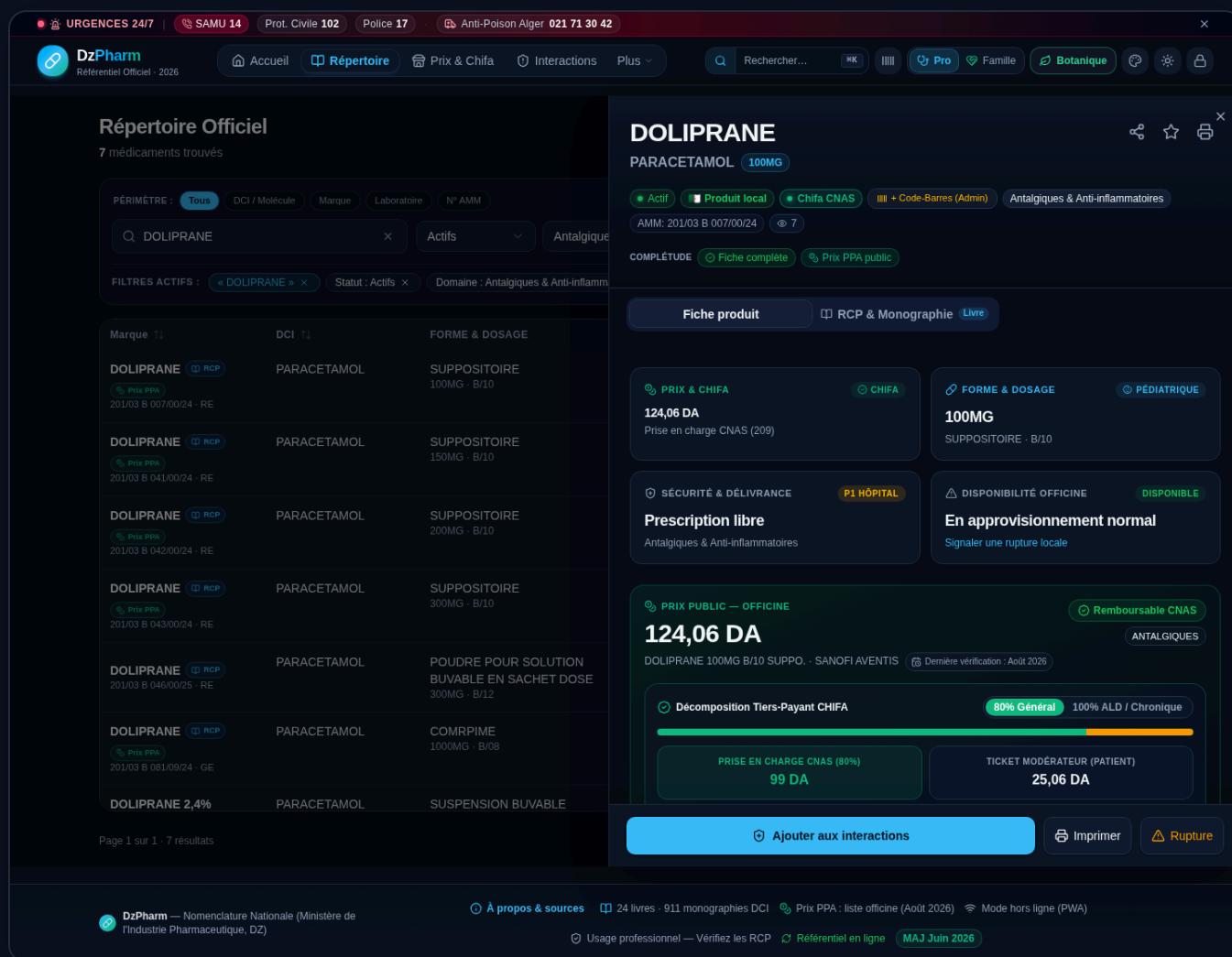


Figure 5.1 — Medication detail modal for DOLIPRANE open over the Répertoire view. The URL has not changed — it remains `/?q=DOLIPRANE&status=ACTIF&domain=Antalgiques+%26+Anti-inflammatoires`. The modal is rendered via `useState`, not via the router.

THE STATE-LOSS SCENARIO

Refresh kills the modal

Step 1: User searches DOLIPRANE, opens the medication detail modal.

Step 2: User reads the RCP, scrolls through the 60 equivalent medications.

Step 3: User accidentally refreshes (or the browser auto-refreshes on a Vercel deployment).

Step 4: The dashboard home loads. The URL still has `?q=DOLIPRANE`, but the user must click "Répertoire" again, then click the DOLIPRANE row again to return to where they were.

Result: ~15 seconds of friction, every time the user refreshes on a modal.

THE FIX · INTERCEPTING ROUTES (NEXT.JS APP ROUTER)

The Next.js App Router has a built-in pattern for exactly this problem: **intercepting routes**. The medication detail can be both a real URL (`/medicament/[slug]`) AND a modal that overlays the current

view without a full page navigation. The pattern uses three files:

```
// File structure
app/
  medicament/
    [slug]/
      page.tsx      # Direct URL access – full page render
    @modal/
      (.)medicament/
        [slug]/
          page.tsx    # Intercepted – renders as modal over current view
      layout.tsx      # Root layout renders both slot and modal slot

// app/@modal/(.)medicament/[slug]/page.tsx
import { MedicationDetailModal } from "@components/medications/modal";
import { getMedication } from "@lib/data";

export default async function InterceptedMedicationModal({ params }) {
  const med = await getMedication(params.slug);
  return <MedicationDetailModal medication={med} />;
}

// app/medicament/[slug]/page.tsx (the direct-access version)
import { MedicationDetailPage } from "@components/medications/page";
export default async function MedicationPage({ params }) {
  const med = await getMedication(params.slug);
  return <MedicationDetailPage medication={med} />; // Same content, no modal chrome
}
```

With this pattern in place, clicking a medication row in the Répertoire calls `router.push("/medicament/doliprane-500mg")`, which is intercepted by the `(.)` matcher and rendered as a modal — but the URL changes. Refresh now loads the full-page version (still the same content, just no overlay). Back button closes the modal. The URL is shareable. All four state-loss vectors are closed with one pattern.

5.2 Drug Interaction List

The drug interaction checker accepts a list of medications via a combobox search, plus a free-text "Contexte patient" input (age, pathologies, pregnancy). The "Analyser les interactions" button is disabled until at least one drug is added. Five pre-set example pairs are offered as quick-start buttons.

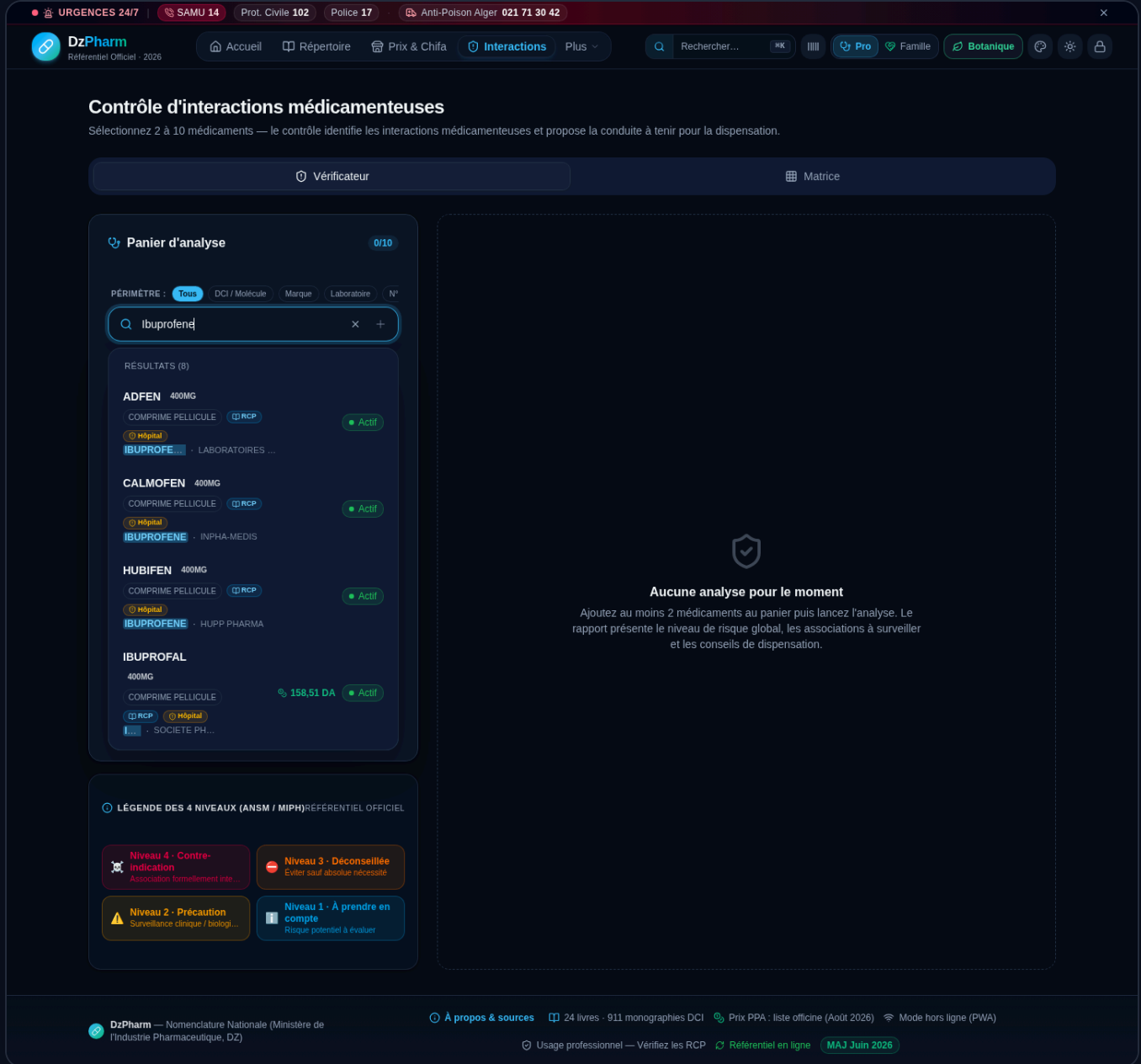


Figure 5.2 — Interactions checker with Paracétamol and Ibuprofène added. The URL has not changed — neither drug is reflected in the query string. The list lives in component `useState`. Refresh would wipe both drugs.

THE STATE-LOSS SCENARIO

Refresh kills the drug list

- Step 1:** Doctor builds up a 6-drug interaction list (5 minutes of search + click work).
- Step 2:** Doctor fills the patient context field (age 65, diabetic, on metformin).
- Step 3:** Doctor accidentally refreshes, or the browser crashes, or the session times out.
- Step 4:** The drug list is gone. The patient context is gone. The doctor must rebuild everything.
- Result:** 5–10 minutes of rework, plus the cognitive cost of remembering the list.

THE FIX · URL-STATE FOR THE DRUG LIST

The drug list should be serialized into the URL as a comma-separated list of medication IDs. The patient context should be a URL parameter too. This makes the entire interaction check shareable — a doctor can send a colleague a URL that opens the exact same analysis.

```
// Hook for syncing drug list to URL
import { useSearchParams, useRouter, usePathname } from "next/navigation";
import { useCallback } from "react";

function useInteractionListState() {
  const searchParams = useSearchParams();
  const router = useRouter();
  const pathname = usePathname();

  // Read drug IDs from URL: /interactions?drugs=9775,8421,1523
  const drugIds = (searchParams.get("drugs") || "")
    .split(",")
    .filter(Boolean)
    .map(Number);

  const patientContext = searchParams.get("ctx") || "";

  const setDrugs = useCallback((newIds) => {
    const params = new URLSearchParams(searchParams);
    if (newIds.length === 0) {
      params.delete("drugs");
    } else {
      params.set("drugs", newIds.join(","));
    }
    router.replace(`/interactions?${params}`, { scroll: false });
  }, [router, searchParams]);

  const setPatientContext = useCallback((ctx) => {
    const params = new URLSearchParams(searchParams);
    if (!ctx) params.delete("ctx");
    else params.set("ctx", ctx);
    router.replace(`/interactions?${params}`, { scroll: false });
  }, [router, searchParams]);

  return { drugIds, patientContext, setDrugs, setPatientContext };
}
```

With this hook, the URL becomes `/interactions?drugs=9775,8421,1523&ctx=65+ans%2C+diabetique`. Refresh preserves everything. Back/forward navigates through interaction-list states. A doctor can send a colleague a URL that opens the exact same analysis. The state-loss problem is solved with ~50 lines of code.

— CHAPTER 06

Scroll, Refresh & Cross-Tab Behavior

Three smaller state behaviors that affect perceived quality: scroll restoration on back/forward, refresh recovery, and cross-tab synchronization. One passes, one is partial, one fails.

6.1 Scroll Restoration · Pass

`history.scrollRestoration` is set to `"auto"` (the browser default). Tested by scrolling 2500px, pushing a new history entry, scrolling another 1000px, then pressing Back — `window.scrollToY` returned

to the original 209px position. The browser handles scroll memory correctly without any application code.

RECOMMENDATION · SET SCROLLRESTORATION EXPLICITLY

Even though `"auto"` works, set it explicitly in the root layout's `useEffect` : `if ("scrollRestoration" in history) history.scrollRestoration = "auto";`. This documents the intent and prevents breakage if a future Next.js version changes the default. Cost: 1 line of code.

6.2 Refresh Recovery · Partial

After refresh on `/?q=DOLIPRANE&status=ACTIF&domain=Antalgiques+%26+Anti-inflammatoires`, the URL is preserved (good), but the active view is not — the dashboard home loads first, and the user must click "Répertoire" to return to the filtered view (bad). The state is technically preserved but the user experience is broken.

Refresh behavior diagnosis

The URL contains the search and filter state. The Zustand store contains the user data. But the *active view* is in component state only, and component state does not survive refresh. The fix is the routing migration (Feature #1) — once each view lives at its own URL, refresh loads that view directly.

6.3 Cross-Tab Synchronization · Fail

A second browser session was opened to the same URL. The second tab redirected to `/login` (the session is not shared via cookies — see Finding 6.4 below). The `localStorage` in the second tab was empty. No `BroadcastChannel` usage was detected. No `window.addEventListener("storage")` listener was observed.

FINDING 6.4 · NO COOKIES, NO SESSION SHARING

The platform uses zero cookies. Authentication state is stored in `sessionStorage["dzpharm_session"] = "active"`. `sessionStorage` is per-tab — opening a new tab starts a fresh session. This is why the second tab redirected to login.

No cross-tab sync · Real-world impact

Scenario: A pharmacist has DzPharm open in two tabs (one for the Répertoire, one for the Interactions checker). They add a medication to favorites in Tab 1. Tab 2 does not see the update until refresh.

Scenario: A doctor is in a Copilot conversation in Tab 1. They open a new tab to search for a related medication. When they return to Tab 1, the Copilot conversation is still there (component state), but if they had refreshed Tab 2, Tab 1's Copilot conversation would also be wiped (because the Zustand store would have been overwritten).

Scenario: A pharmacist leaves DzPharm open at work, goes home, opens DzPharm there. Their favorites and recently-viewed do not sync (different machine, different `localStorage`). This requires server-side accounts (Feature #8).

THE FIX · STORAGE-EVENT LISTENER + BROADCASTCHANNEL

For same-machine, multi-tab sync (no accounts required), add a `storage` event listener that rehydrates the Zustand store when another tab writes to localStorage:

```
// In the Zustand store setup
import {{ create }} from "zustand";
import {{ persist, createJSONStorage }} from "zustand/middleware";

export const useStore = create(
  persist(
    (set, get) => ({{ /* ... state and actions ... */ }}),
    {{
      name: "dzpharm-store",
      version: 3,
      storage: createJSONStorage(() => localStorage),

      // Listen for cross-tab updates
      onRehydrateStorage: () => (state) => {{
        if (typeof window !== "undefined") {{
          window.addEventListener("storage", (e) => {{
            if (e.key === "dzpharm-store" && e.newValue) {{
              try {{
                const newState = JSON.parse(e.newValue);
                useStore.setState(newState.state, true); // replace: true
              }} catch (err) {{
                console.warn("Failed to sync store from another tab", err);
              }}
            }}
          }}
        }}
      }}
    }}
  )
);
```

With this 10-line addition, when Tab A adds a favorite, Tab B sees the update within ~50ms (the latency of the storage event). No server required. This is a one-day implementation.

— CHAPTER 07

Fix Plan · Prioritized by Impact vs. Effort

Six concrete fixes, ranked by impact-to-effort ratio. The first three are mechanical same-day changes; the last three require coordination with the routing migration from Feature #1.

#	FIX	IMPACT	EFFORT	DEPENDENCIES
1	Sync interactions drug list + patient context to URL	HIGH	S (1 day)	None — uses existing <code>useSearchParams</code>
2	Persist Copilot history to Zustand store (v3 migration)	HIGH	S (1 day)	None
3	Add cross-tab sync via storage event listener	MED	S (1 day)	None
4	Migrate medication detail to intercepting route	HIGH	M (3 days)	Feature #1 routing migration
5	Cap <code>recentlyViewed</code> at 10 entries (currently 1)	MED	XS (2 hours)	None
6	Clean up legacy localStorage keys (palette, design_mode)	LOW	XS (1 hour)	None

7.1 The Same-Day Three (Fixes #1, #2, #3)

These three fixes can ship together in a single PR with roughly 150 lines of code. Together they close the four critical state gaps without requiring the routing migration:

- ✓ Fix #1 (drug list in URL) closes Gap 3 — interactions list now survives refresh and is shareable.
- ✓ Fix #2 (Copilot history in Zustand) closes Gap 4 — Copilot conversations now survive refresh and tab close.
- ✓ Fix #3 (cross-tab sync) closes the multi-tab sync gap — favorites, Copilot history, and interactions list now sync across tabs.

7.2 The Migration-Dependent Three (Fixes #4, #5, #6)

These three fixes are lower-priority or depend on the routing migration from Feature #1:

- ▶ Fix #4 (intercepting routes for medication detail) closes Gap 2 — but requires the routing migration to ship first.
- ▶ Fix #5 (raise `recentlyViewed` cap) is a one-line change but low-impact.
- ▶ Fix #6 (clean up legacy keys) is housekeeping — schedule for a future cleanup sprint.

7.3 Recommended Sequencing

Week 1 • Ship the Same-Day Three

Open a single PR that adds (a) the `useInteractionListState` hook, (b) the `copilotHistory` slice in the Zustand store with v3 migration, (c) the storage event listener in `onRehydrateStorage`. Test each in isolation, then merge together. Estimated effort: 1 engineer × 3 days.

Week 2 • Coordinate with Routing Migration

Once Feature #1 (routing migration) ships its Phase 1 (the four primary views become real routes), the medication detail intercepting-route pattern (Fix #4) becomes possible. Schedule Fix #4 to ship alongside the routing migration's Phase 3 (dynamic medication pages).

Week 3+ · Cleanup

Schedule Fix #5 (raise cap) and Fix #6 (cleanup legacy keys) as part of a state-management polish sprint. These are housekeeping tasks that can ship in any low-stakes PR.

TOTAL EFFORT ESTIMATE

Same-Day Three: 3 days · Migration-Dependent Three: 1 week (parallel to Feature #1) · Cleanup: 1 day.
Total: ~2 weeks of focused state-management work, much of it parallel to the routing migration.

CHAPTER 08

Summary & Next Steps

DzPharm's state-management architecture is **better-built than most** — the Zustand-persisted store is the right pattern, the schema versioning is correct, and the URL-state for search and filters is properly wired. But the architecture is **incomplete** in four high-impact areas, each of which can be closed with a small, mechanical fix.

8.1 The Asset Inventory

Zustand + persist + version 2

Senior-level state engineering. The right library, the right middleware, the right schema-versioning discipline. Rare in production apps.

URL-state for search and 3 filters

The team has correctly wired `useSearchParams` for `?q=`, `?status=`, and `?domain=`. The pattern just needs to be extended to other state.

Scroll restoration works by default

No application code needed — `history.scrollRestoration = "auto"` handles it. (Recommend documenting this explicitly to prevent regression.)

Theme stored separately from Zustand

The `theme` localStorage key is correctly outside the Zustand store — it must be readable before React hydrates to avoid FOUC. The team understood this trade-off.

8.2 The Gap Inventory

Active view not in URL

Refresh on `/?q=...` loads dashboard home, not the Répertoire view that the URL implies. Closed by Feature #1 routing migration.

Medication detail modal not in URL

Cannot share, bookmark, or refresh-survive a medication detail view. Closed by Next.js intercepting-route pattern (Fix #4).

Interactions drug list not in URL or store

5–10 minutes of work lost on refresh. Closed by the `useInteractionListState` hook (Fix #1, 1 day of work).

Copilot history not persisted

Every AI conversation lost on refresh. Re-asks cost money (Gemini 3.8 Flash) and time. Closed by adding `copilotHistory` slice to Zustand (Fix #2, 1 day of work).

No cross-tab sync

Favorites and history updates in one tab do not appear in another until refresh. Closed by adding a `storage` event listener in `onRehydrateStorage` (Fix #3, 1 day of work).

8.3 The Strategic Recommendation

Ship the Same-Day Three (Fixes #1, #2, #3) as a single PR in week 1. This closes three of the four critical state gaps with roughly 150 lines of code and no architectural changes. Coordinate Fix #4 (intercepting routes for medication detail) with the Feature #1 routing migration in week 2. Schedule the cleanup work (Fixes #5, #6) for a low-stakes sprint.

The most important insight from this audit: **the team has already done the hard part**. Zustand with persist + schema versioning is the right foundation. `useSearchParams` for search and filters is the right pattern. The remaining work is mechanical extension of an existing pattern, not architectural redesign. This is one of the rare cases where a state-management audit concludes with "ship 3 small PRs in a week and you're done."

THE BOTTOM LINE

DzPharm's state architecture is 70% excellent and 30% incomplete. The 30% can be closed in 2 weeks of focused work, most of it parallel to the routing migration from Feature #1. After that, the platform has the same state-management quality as Linear, Vercel, or Notion — shareable, bookmarkable, refresh-safe, and cross-tab-synced.

Immediate Next Steps · This Week

1. Open a PR adding `useInteractionListState` hook (Fix #1, ~50 LOC).
2. Open a PR adding `copilotHistory` slice to the Zustand store with a v2 → v3 migration (Fix #2, ~80 LOC).
3. Open a PR adding the storage event listener for cross-tab sync (Fix #3, ~10 LOC).
4. Coordinate Fix #4 with the Feature #1 routing migration team — the medication detail intercepting route is a natural fit for Phase 3 of that migration.
5. Schedule Fixes #5 and #6 for a future polish sprint.

state

70% excellent, 30% mechanical. Two weeks of small PRs to close every gap.

8

CHAPTERS

8

STATE CATEGORIES

6

FIXES PROPOSED

2

WEEKS TO SHIP
